

Ghid pentru rezolvarea problemei cu Transformer

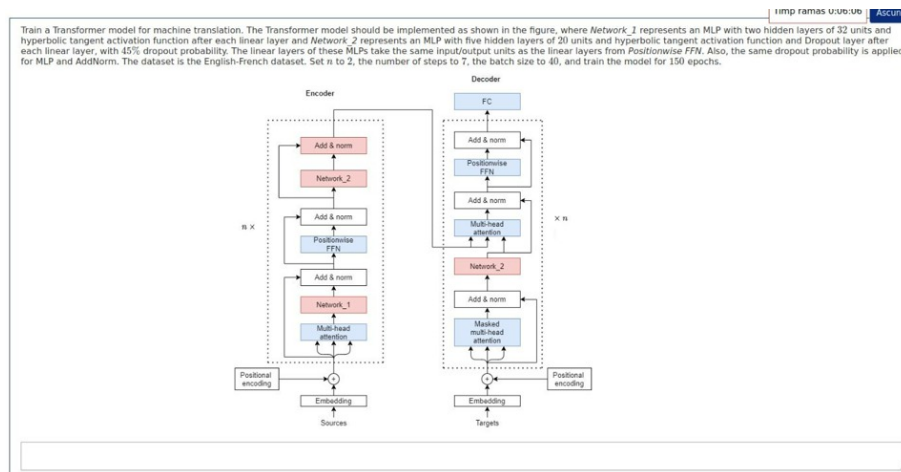


Figura 1: Problema Transformer

- Se copiaza toate celulele de cod pe la Laboratorul 11, incepand cu functia `sequence_mask` pana jos de tot, pana la celula:

Listing 1: Celula finala pana la care sa copiat

```

1 num_hiddens, num_layers, dropout, batch_size, num_steps = 32, 2,
  0.1, 64, 10
2 lr, num_epochs, device = 0.005, 200, try_gpu()
3 ffn_num_input, ffn_num_hiddens, num_heads = 32, 64, 4
4 key_size, query_size, value_size = 32, 32, 32
5 norm_shape = [32]
6
7 train_iter, src_vocab, tgt_vocab = load_data_nmt(batch_size,
  num_steps)
8
9 encoder = TransformerEncoder(

```

```

10     len(src_vocab), key_size, query_size, value_size, num_hiddens,
11     norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
12     num_layers, dropout)
13 decoder = TransformerDecoder(
14     len(tgt_vocab), key_size, query_size, value_size, num_hiddens,
15     norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
16     num_layers, dropout)
17 net = EncoderDecoder(encoder, decoder)
18 train_loss_all = train_seq2seq(net, train_iter, lr, num_epochs,
    tgt_vocab, device) #1 min

```

- Celulele de mai sus ne ofera tot mai putin acele retele noi pe care trebuie sa le implementam, Network1 si Network2. Asadar, pentru inceput, voi crea o clasa noua pentru Network1 si Network2 si voi incerca sa le implementez conform cerintelor. Incep cu Network1. Imi spune ca e un MLP cu 2 straturi ascunse, fiecare strat ascuns cu 32 neuroni, si o tangenta hiperbolica ca functie de activare dupa fiecare strat liniar. Creez clasa asa:

Listing 2: Network1

```

1 class Network1(nn.Module):
2     def __init__(self, input_channels, output_channels):
3         super().__init__()
4         self.network = nn.Sequential(
5             nn.Linear(input_channels, 32),
6             nn.Tanh(),
7             nn.Linear(32, 32),
8             nn.Tanh(),
9             nn.Linear(32, output_channels),
10            nn.Tanh()
11        )
12
13    def forward(self, X):
14        return self.network(X)

```

- Mai apoi, fac clasa pentru Network2. La fel ca Network1, doar ca acum are 5 straturi ascunse, 20 neuroni fiecare strat, aceeași funcție de activare dar se mai adauga dropout de 45% dupa fiecare strat ascuns:

Listing 3: Network2

```

1 class Network2(nn.Module):
2     def __init__(self, input_channels, output_channels):
3         super().__init__()
4         self.network = nn.Sequential(
5             nn.Linear(input_channels, 20),
6             nn.Tanh(),
7             nn.Dropout(0.45),
8             nn.Linear(20, 20),
9             nn.Tanh(),
10            nn.Dropout(0.45),
11            nn.Linear(20, 20),

```

```

12         nn.Tanh(),
13         nn.Dropout(0.45),
14         nn.Linear(20,20),
15         nn.Tanh(),
16         nn.Dropout(0.45),
17         nn.Linear(20,20),
18         nn.Tanh(),
19         nn.Dropout(0.45),
20         nn.Linear(20,output_channels),
21         nn.Tanh(),
22         nn.Dropout(0.45)
23     )
24
25     def forward(self, X):
26         return self.network(X)

```

- Am definit *Network1* si *Network2*. Acum trebuie sa modificam encoder-ul (partea stanga) si decoder-ul (partea dreapta) din Transformer ca sa introducem noile retele. Ce trebuie sa stim este ca, daca din poza am elimina *network1* si *network2*, respectiv *add norm*-urile pe care ele le introduc, obtinem codul din laborator. Asadar, noi trebuie doar sa integram *Network1* si *Network2* in acel pipeline. E singurul lucru in arhitectura pe care il modificam, iar aceste modificari se fac in *EncoderBlock* si *DecoderBlock*. Acestea reprezinta blocurile de baza pentru Encoder si, respectiv, Decoder. Hai sa incepem cu Encoder-ul, adica *EncoderBlock*. Prima data voi introduce *Network1* si *Network2* in constructorul de la *EncoderBlock*, asa:

Listing 4: EncoderBlock modificat

```

1 class EncoderBlock(nn.Module):
2     """Transformer encoder block."""
3     def __init__(self, key_size, query_size, value_size,
4                 num_hiddens,
5                 norm_shape, ffn_num_input, ffn_num_hiddens,
6                 num_heads,
7                 dropout, use_bias=False, **kwargs):
8         super(EncoderBlock, self).__init__(**kwargs)
9         self.attention = MultiHeadAttention(
10             key_size, query_size, value_size, num_hiddens,
11             num_heads, dropout,
12             use_bias)
13         self.addnorm1 = AddNorm(norm_shape, dropout)
14         self.ffn = PositionWiseFFN(
15             ffn_num_input, ffn_num_hiddens, num_hiddens)
16         self.addnorm2 = AddNorm(norm_shape, dropout)
17         self.network1 = Network1(ffn_num_input, num_hiddens)
18         self.network2 = Network2(ffn_num_input, num_hiddens)
19         self.addnorm3 = AddNorm(norm_shape, dropout)
20
21     def forward(self, X, valid_lens):
22         X1 = self.addnorm1(X, self.network1(self.attention(X, X, X,
23             valid_lens)))
24         X2 = self.addnorm2(X1, self.ffn(X1))
25         return self.addnorm3(X2, self.network2(X2))

```

- *Daca ne uitam la graficul de la encoder, fix asta se intampla: rezultatul de la atentie este transmis in **network1**, iesirea de la **network1** este adunata inapoi cu intrarea originala, tot acest output intra in FFN, dupa avem iara o conexiune reziduala prin **addnorm2**, dupa rezultatul acelei conexiuni reziduale este transmis in **network2** si dupa iara o conexiune reziduala. Simplu, nu-i asa? Atentie la add norm, in caz ca trebuie adaugat unul nou. In acest caz trebuie, si am creat un **addnorm3** pentru acea conexiune reziduala de la **Network2**. Orice add norm vazut pe grafic va fi reprezentat individual.*
- *De ce **Network1** si **Network2** iau **ffn_num_input** neuroni ca intrare? Deoarece enuntul zice ca intrarea/iesirea lor este data de intrarea/iesirea lui **PositionWiseFFN**. Daca ne uitam la **PositionWiseFFN**, el ia **ffn_num_input** ca intrare si da ca iesire **num_hiddens** (**ffn_num_hiddens** reprezinta o dimensionalitate intermediara pe care el il foloseste ca sa ajunga la output-ul final de **num_hiddens**).*
- *Acum trebuie sa facem acelasi lucru si pentru **DecoderBlock**. Voi adauga **Network1** si **Network2** in clasa de la **DecoderBlock**:*

Listing 5: DecoderBlock modificat

```

1 class DecoderBlock(nn.Module):
2     # The 'i'-th block in the decoder
3     def __init__(self, key_size, query_size, value_size,
4         num_hiddens,
5         norm_shape, ffn_num_input, ffn_num_hiddens,
6         num_heads,
7         dropout, i, **kwargs):
8         super(DecoderBlock, self).__init__(**kwargs)
9         self.i = i
10        self.attention1 = MultiHeadAttention(
11            key_size, query_size, value_size, num_hiddens,
12            num_heads, dropout)
13        self.addnorm1 = AddNorm(norm_shape, dropout)
14        self.attention2 = MultiHeadAttention(
15            key_size, query_size, value_size, num_hiddens,
16            num_heads, dropout)
17        self.addnorm2 = AddNorm(norm_shape, dropout)
18        self.ffn = PositionWiseFFN(ffn_num_input, ffn_num_hiddens,
19            num_hiddens)
20        self.addnorm3 = AddNorm(norm_shape, dropout)
21        self.network2 = Network2(ffn_num_input, num_hiddens)
22
23    def forward(self, X, state):
24        enc_outputs, enc_valid_lens = state[0], state[1]
25        # During training, all the tokens of any output sequence
26        # are processed
27        # at the same time, so 'state[2][self.i]' is 'None' as
28        # initialized.
29        # When decoding any output sequence token by token during
30        # prediction,
31        # 'state[2][self.i]' contains representations of the
32        # decoded output at

```

```

25     # the 'i'-th block up to the current time step
26     if state[2][self.i] is None:
27         key_values = X
28     else:
29         key_values = torch.cat((state[2][self.i], X), axis=1)
30     state[2][self.i] = key_values
31     if self.training:
32         batch_size, num_steps, _ = X.shape
33         # Shape of 'dec_valid_lens' is: ('batch_size', '
34         # num_steps')
35         dec_valid_lens = torch.arange(
36             1, num_steps + 1, device=X.device).repeat(
37                 batch_size, 1)
38     else:
39         dec_valid_lens = None
40
41     # Self-attention
42     X2 = self.attention1(X, key_values, key_values,
43                          dec_valid_lens)
44     Y = self.addnorm1(X, X2)
45     Y2 = self.network2(Y)
46     # Encoder-decoder attention
47     Y3 = self.attention2(Y2, enc_outputs, enc_outputs,
48                          enc_valid_lens)
49     Z = self.addnorm2(Y2, Y3)
50     return self.addnorm3(Z, self.ffn(Z)), state

```

- *Mai sus, am adaugat chestii dupa comentariul cu # Self-attention*
- *Asta e aproape tot! Mai trebuie sa modificam parametri pe care ii zice: n (number of layers), number of steps, batch size si number of epochs. Asta se realizeaza strict in ultima celula, asa:*

Listing 6: Parametri ajustati

```

1 num_hiddens, num_layers, dropout, batch_size, num_steps = 32, 2,
2   0.45, 40, 2 # Aici modific
3 lr, num_epochs, device = 0.005, 150, try_gpu()
4 ffn_num_input, ffn_num_hiddens, num_heads = 32, 64, 4
5 key_size, query_size, value_size = 32, 32, 32
6 norm_shape = [32]

```

- *Cam asta e tot la problema cu Transformere. Nu e greu!*